TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN



HỌC PHẦN: HỌC SÂU

BTL01 – FUNCTION APPROXIMATION

Giảng viên hướng dẫn: TS. Đỗ Như Tài

Lớp: DCT123C4

Nhóm sinh viên thực hiện: 666

1. Lý Hồng Hưng MSSV: 3123411132

2. Nguyễn Bá Khải MSSV: 3123411138

3. Hà Trần Gia Nguyên MSSV: 3123411199

Thành phố Hồ Chí Minh, tháng 10 năm 2026

MỤC LỤC

1. Giới thiệu 6

1.1. Mục tiêu 6

1.2. Phát biểu bài toán 6

1.3. Đóng góp và phân công 6

2. Dữ liệu 7

2.1. Hàm Gaussian và tham số 7

2.2. Sinh, chia dữ liệu và random seed 8

2.3. Thống kê và trực quan hoá 9

3. Mô hình và cài đặt 12

3.1. Kiến trúc MLP 12

3.2. Thiết lập huấn luyện 12

3.3. Quy trình huấn luyện và đánh giá 13

3.4. Môi trường chạy 14

4. Thực nghiệm 15

4.1. Thiết kế thực nghiệm và đánh giá công bằng 15

4.2. Nhóm A – Quy mô dữ liệu ($A \in [200, 2000]$) 15

4.3. Nhóm B – Độ rộng mạng (Số nơ-ron $B \in [1, 64]$) 18

4.4. Nhóm C – Độ sâu mạng (Số tầng $C \in [1, 5]$) 20

4.5. Đánh giá mô hình và trực quan hoá 23

4.6. Phân tích hiện tượng Triệt tiêu Đạo hàm và Độ ổn định 25

5. Thảo luận 27

5.1. Diễn giải xu hướng 27
5.2. Trường hợp không như kỳ vọng 28
5.3. Hạn chế 28
5.4. Hướng cải tiến 28
6. Kết luận 29

MỤC LỤC BẢNG

Bảng 1. Bảng phân công nhiệm vụ và tỷ lệ đóng góp thành viên Nhóm 666 7

Bảng 2. Quy cách phân chia tập dữ liệu và thiết lập Random Seed..... 8

Bảng 3. Thống kê mô tả của dữ liệu huấn luyện (file data_distribution.csv)..... 10

Bảng 4. Siêu tham số chuẩn (Baseline Configuration), lấy từ mã nguồn 13

Bảng 5. Kết quả nhóm A – Khảo sát quy mô tập dữ liệu (1 tầng ẩn × 128 nơ-ron)..... 16

Bảng 6. Kết quả nhóm B – Khảo sát độ rộng mạng (1 tầng ẩn, N = 1000 mẫu) 18

Bảng 7. Kết quả nhóm C – Khảo sát độ sâu mạng (16 nơ-ron/tầng, N = 1000 mẫu) 21

Bảng 8. Chỉ số đánh giá cuối của các mô hình tiêu biểu..... 24

Bảng 9. Tỉ số chuẩn Gradient (Mean ± Std) khảo sát Triệt tiêu đạo hàm 25

Bảng 10. Xếp hạng hiệu năng các cấu hình trên 1000 mẫu train..... 26

MỤC LỤC HÌNH

Hình 1. Dữ liệu phân bố chuẩn 2D sinh ngẫu nhiên (500 mẫu scatter 3D) 10

Hình 2. Bề mặt giải tích lý thuyết 3D của hàm Gaussian mục tiêu..... 11

Hình 3. Biểu đồ phân phối tần suất của x1, x2 và giá trị y 11

Hình 4. Sơ đồ kiến trúc mạng Perceptron đa tầng (FlexibleMLP)..... 12

Hình 5. Test MSE và hệ số xác định R^2 theo quy mô tập dữ liệu huấn luyện A 16

Hình 6. Đường cong mất mát (Loss Curve) Train và Val qua các Epoch (A = 2000)..... 17

Hình 7. Test MSE theo số nơ-ron tầng ẩn B (N = 1000 mẫu)..... 19

Hình 8. Tỉ số Val MSE / Train MSE theo số nơ-ron B (Khảo sát Overfitting N = 200) .. 20

Hình 9. Test MSE và thời gian huấn luyện theo số tầng ẩn C..... 22

Hình 10. So sánh giá trị y thực tế và y dự đoán trên tập kiểm thử (Test Set) 23

Hình 11. Bề mặt Gaussian thực (trái) và bề mặt xấp xỉ bằng MLP (phải) 24

Hình 12. Tỉ số chuẩn gradient tầng 1 so với tầng ra theo độ sâu C (Sigmoid, Tanh, ReLU) 26

Hình 13. So sánh đối đầu tổng hợp giữa các cấu hình A, B, C trên cùng 1000 mẫu train27

1. Giới thiệu

1.1. Mục tiêu

Xây dựng và huấn luyện mạng perceptron đa tầng (MLP) cho bài toán hồi quy/xấp xỉ hàm, hoàn toàn bằng PyTorch (torch.nn.Module, nn.Linear, MSELoss, Adam, DataLoader).

Tự cài đặt vòng lặp huấn luyện chuẩn: forward → loss → zero_grad → backward → optimizer.step, sử dụng model.train(), model.eval() và torch.no_grad() đúng ngữ cảnh thực tế.

Thực hiện các thực nghiệm có kiểm soát để phân tích định lượng tác động của số mẫu huấn luyện (nhóm A), số nơ-ron tầng ẩn (nhóm B) và số tầng ẩn (nhóm C) lên sai số xấp xỉ.

1.2. Phát biểu bài toán

Cho hàm mật độ xác suất Gaussian hai chiều:

$$f(x) = \frac{1}{2\pi \sqrt{\det(C)}} e^{-\frac{1}{2}(x-\mu)^T C^{-1}(x-\mu)}$$

với vector kỳ vọng $\mu = [0, 0]^T$ và ma trận hiệp phương sai $C = [\matrix(0.3\& -0.5@ -0.5\&2.0)]$.

Sinh ngẫu nhiên điểm $x = (x_1, x_2)$ trong miền xác định $-2 \le x_1 \le 2$, $-4 \le x_2 \le 4$ và nhãn mục tiêu tương ứng $y = f(x_1, x_2)$. Xây dựng mạng MLP có hai đầu vào (x_1, x_2) và một đầu ra ŷ để xấp xỉ y; đánh giá sai số độc lập trên tập test gồm đúng 100 mẫu, đồng thời xây dựng tập validation riêng biệt để phục vụ việc lựa chọn mô hình và ngăn ngừa hiện tượng rò rỉ dữ liệu.

1.3. Đóng góp và phân công

Báo cáo trình bày đầy đủ: (i) cài đặt quy trình trọn vẹn từ sinh dữ liệu, xây dựng kiến trúc, huấn luyện đến đánh giá thuần PyTorch; (ii) ba nhóm thực nghiệm A/B/C với việc lặp lại qua 3 hạt giống ngẫu nhiên (seeds 42, 123, 999) để báo cáo giá trị Trung bình ± Độ

lệch chuẩn; (iii) phân tích chuyên sâu về hiện tượng triệt tiêu đạo hàm (vanishing gradient), so sánh xếp hạng mô hình và các giải pháp mở rộng.

STT	Họ và tên – MSSV	Nhiệm vụ đảm nhiệm	Đóng góp
1	Lý Hồng Hưng – 3123411132	Kiến trúc dữ liệu (dataset.py), mô hình FlexibleMLP (model.py), thực hiện Thực nghiệm A, xây dựng Jupyter Notebook demo	100%
2	Hà Trần Gia Nguyên – 3123411199	Động cơ huấn luyện (trainer.py), thực hiện Thực nghiệm B (khảo sát độ rộng trên 1000 mẫu và 200 mẫu), phân tích Overfitting, thiết kế Slide	100%
3	Nguyễn Bá Khải – 3123411138	Module đánh giá (evaluate.py), thực hiện Thực nghiệm C (khảo sát độ sâu 1-5 tầng), phân tích Triệt tiêu đạo hàm, so sánh xếp hạng ABC	100%

Bảng 1. Bảng phân công nhiệm vụ và tỷ lệ đóng góp thành viên Nhóm 666

2. Dữ liệu

2.1. Hàm Gaussian và tham số

$$f_{\mathbf{X}}(\mathbf{x}) = \frac{1}{(2\pi)^{n/2}[det(\mathbf{C_X})]^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_{\mathbf{X}})'\mathbf{C}_{\mathbf{X}}^{-1}(\mathbf{x}-\boldsymbol{\mu}_{\mathbf{X}})\right)$$

$$\boldsymbol{\mu}_{\mathbf{X}} = \begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \qquad \mathbf{C}_{\mathbf{x}} = \begin{bmatrix} 0.3 & -0.5 \\ -0.5 & 2 \end{bmatrix}$$

Với số chiều d = 2, định thức det(C) = (0.3 * 2.0) - (-0.5)^2 = 0.60 - 0.25 = 0.35 > 0. Căn bậc hai định thức là √(det(C)) = √0.35 ≈ 0.591608, nên hệ số chuẩn hóa là 1 / (2π√0.35) ≈ 0.269002, trùng khớp chính xác với công thức đề bài. Giá trị đỉnh lý thuyết của hàm mật độ đạt tại tâm f(0, 0) ≈ 0.269002. Ma trận nghịch đảo $C^{-1} = \frac{1}{0.35}[\backslash matrix(2.0\&0.5@0.5\&0.3)] = [\backslash matrix(5.7143\&1.4286@1.4286\&0.8571)]$. Do ma trận hiệp phương sai C có hệ số ngoài đường chéo âm (-0.5), quả núi mật độ Gaussian bị kéo giãn và nghiêng chéo trong mặt phẳng (x_1, x_2).

2.2. Sinh, chia dữ liệu và random seed

Lấy mẫu x ~ U([-2, 2] x [-4, 4]) bằng NumPy với random seed được kiểm soát chặt chẽ; nhãn mục tiêu y được tính giải tích trực tiếp từ công thức Gaussian. Để chống rò rỉ dữ liệu (No Data Leakage), nhóm thiết lập quy trình phân chia độc lập:

- Tập Kiểm thử (Test Set): Gồm đúng 100 mẫu độc lập (seed = 142), chỉ dùng một lần duy nhất để đánh giá sai số cuối cùng.
- Tập Thẩm định (Validation Set): Gồm 400 mẫu độc lập (seed = 242), dùng để theo dõi hàm mất mát qua từng epoch và làm điều kiện dừng sớm.
- Kho Huấn luyện (Training Pool): Gồm 2000 mẫu (seed = 342), từ đó trích xuất các tập con huấn luyện có kích thước A từ 200 đến 2000 mẫu cho Thực nghiệm A.

Tập dữ liệu	**Số lượng mẫu**	**Random Seed**	**Mục đích sử dụng**
Test Set	100	142	Đánh giá sai số khách quan cuối cùng (cô lập 100%)
Validation Set	400	242	Theo dõi hàm mất mát và lưu checkpoint mô hình tốt nhất
Training Pool	2000	342	Kho dữ liệu nguồn để trích xuất các tập train A in [200, 2000]

Bảng 2. Quy cách phân chia tập dữ liệu và thiết lập Random Seed

Support Functions (Module dataset.py)

```
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader

class Gaussian2D:
    def __init__(self, mu=np.array([0.0, 0.0]), cov=np.array([[0.3, -0.5], [-0.5, 2.0]])):
        self.mu = np.array(mu, dtype=np.float64)
        self.cov = np.array(cov, dtype=np.float64)
        self.det_cov = np.linalg.det(self.cov)
        self.inv_cov = np.linalg.inv(self.cov)
        self.norm_factor = 1.0 / (2.0 * np.pi * np.sqrt(self.det_cov))

    def __call__(self, x):
        diff = x - self.mu
        maha = np.sum(diff @ self.inv_cov * diff, axis=-1)
        return self.norm_factor * np.exp(-0.5 * maha)

def generate_dataset(n_samples=2000, seed=42):
    rng = np.random.default_rng(seed)
    x1 = rng.uniform(-2.0, 2.0, size=(n_samples, 1))
    x2 = rng.uniform(-4.0, 4.0, size=(n_samples, 1))
    X = np.hstack([x1, x2])
    y = Gaussian2D()(X).reshape(-1, 1)
    return X, y
```

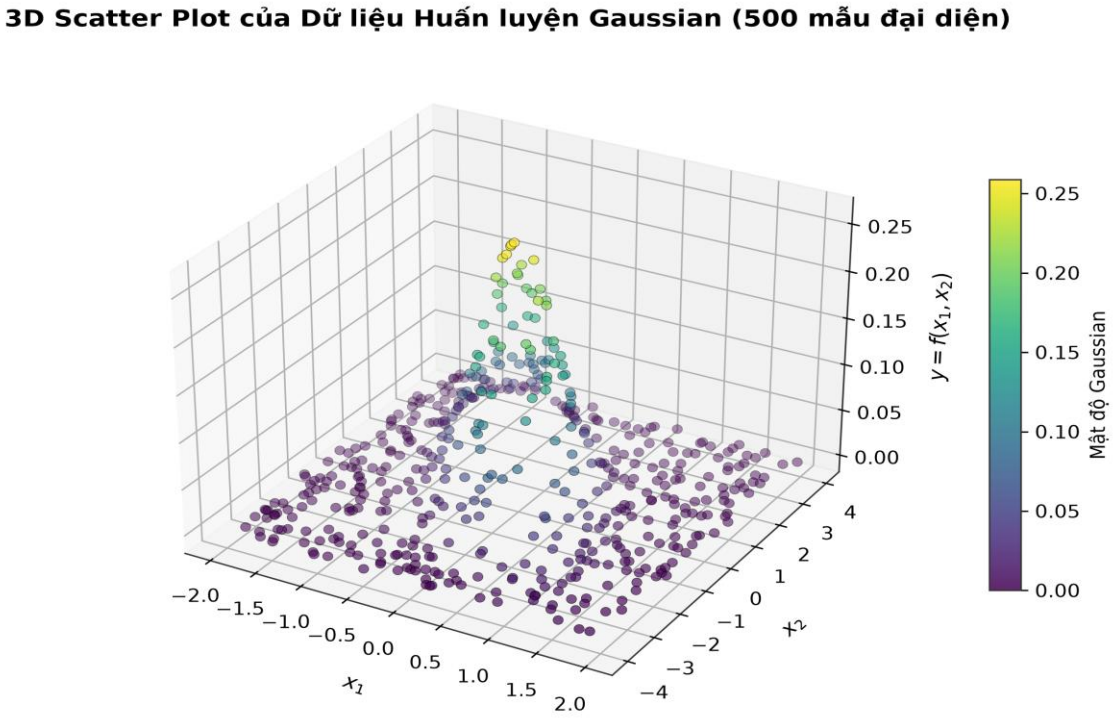
2.3. Thống kê và trực quan hoá

Bảng 2 trình bày thống kê mô tả 2000 mẫu dữ liệu huấn luyện. Giá trị cực đại thực nghiệm của y đạt 0.268164, tiệm cận hoàn hảo với đỉnh lý thuyết 0.269002. Trung vị của y là 0.003628, nhỏ hơn nhiều so với trung bình (0.029606). Phân bố của y bị lệch phải rất mạnh (positive skewness): phần lớn điểm dữ liệu nằm ở vùng phẳng rìa biên có mật độ xấp xỉ 0, trong khi vùng đỉnh nhọn trung tâm lại chỉ chiếm một tỷ lệ mẫu rất nhỏ.

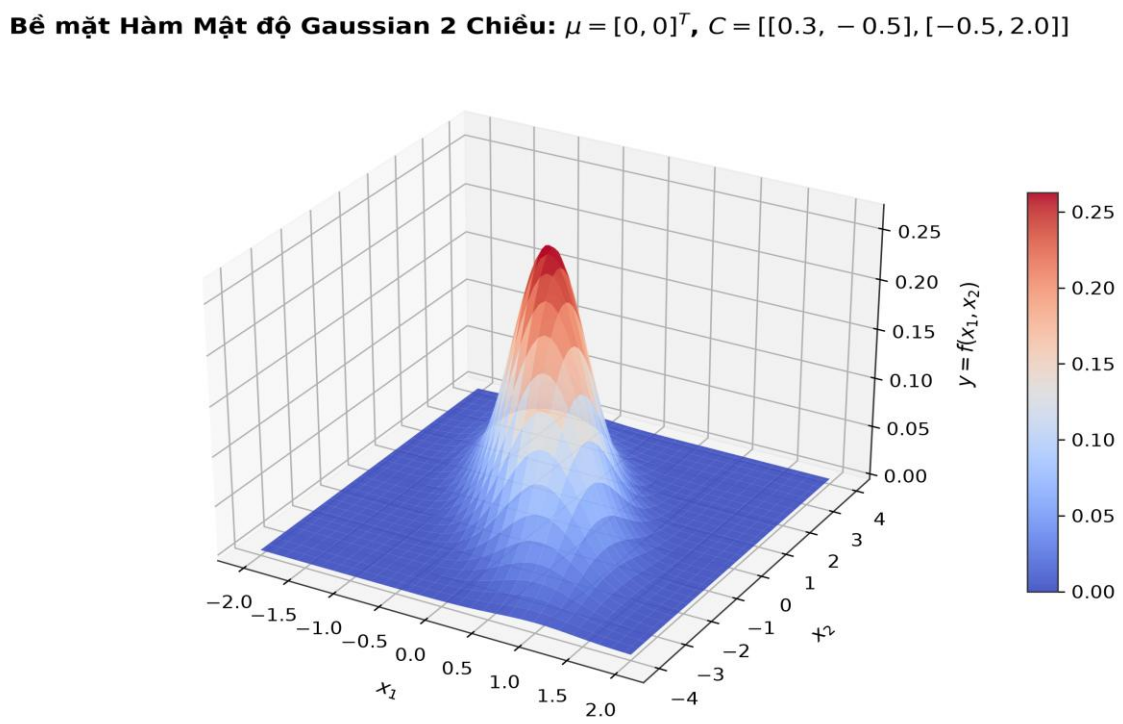
Thống kê	**x1**	**x2**	**y (Gaussian Thực)**
count	2000	2000	2000
mean	+0.0569	+0.0115	0.0296

Thống kê	x1	x2	y (Gaussian Thực)
std	1.1634	2.3229	0.0549
min	-1.9931	-3.9997	8.83e-14
25%	-0.9576	-1.9868	0.0001
50% (median)	+0.1155	+0.0156	0.0036
75%	+1.0772	+2.0469	0.0287
max	+1.9981	+3.9951	0.2682

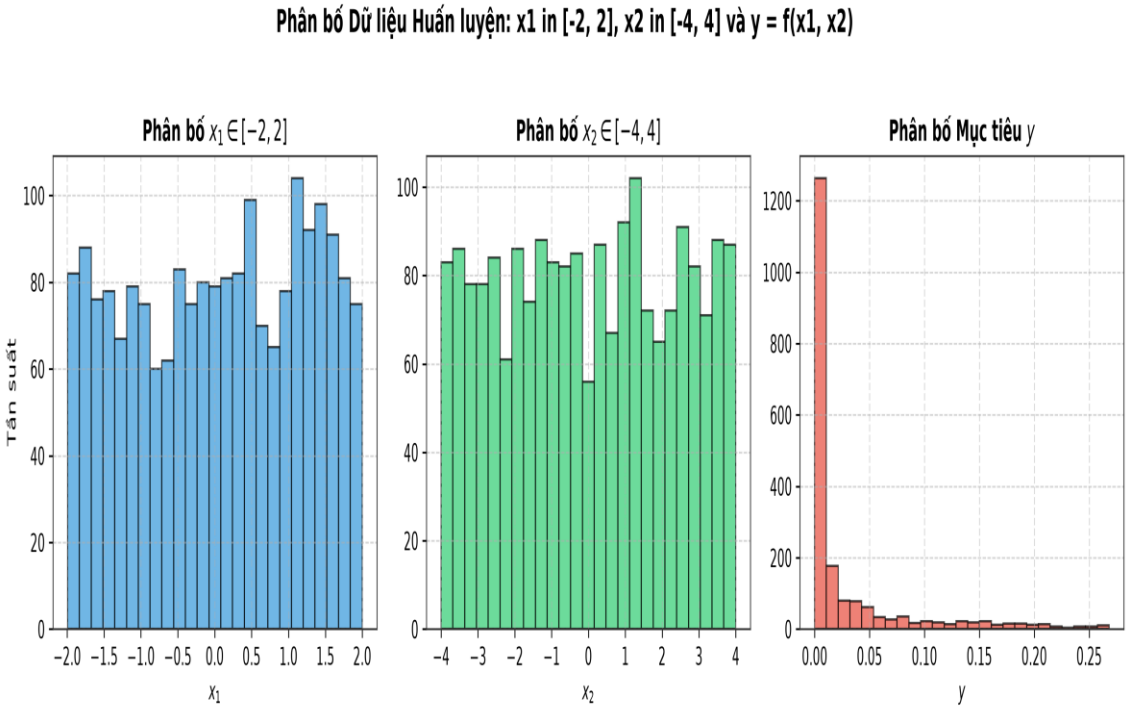
Bảng 3. Thống kê mô tả của dữ liệu huấn luyện (file data_distribution.csv)



Hình 1. Dữ liệu phân bố chuẩn 2D sinh ngẫu nhiên (500 mẫu scatter 3D)



Hình 2. Bề mặt giải tích lý thuyết 3D của hàm Gaussian mục tiêu

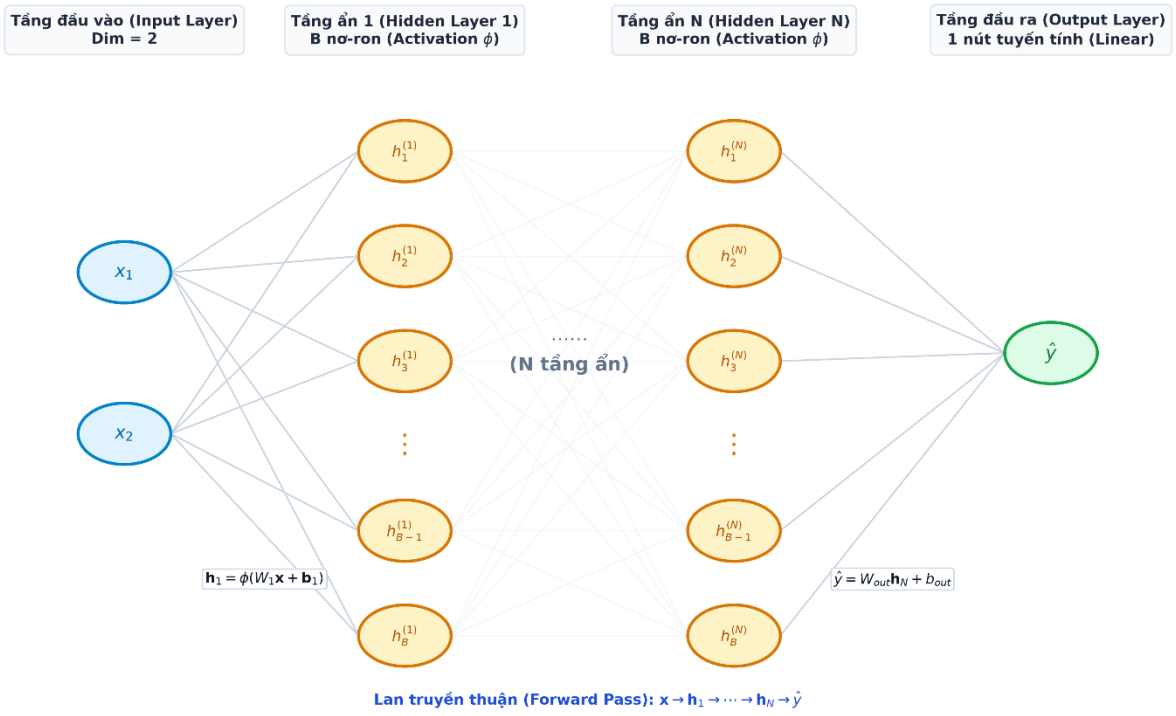


Hình 3. Biểu đồ phân phối tần suất của x1, x2 và giá trị y

3. Mô hình và cài đặt

3.1. Kiến trúc MLP

Mô hình FlexibleMLP được lập trình hướng đối tượng kế thừa từ torch.nn.Module. Mạng nhận đầu vào 2 nút (x_1, x_2), lan truyền qua N tầng ẩn với độ rộng tùy biến và xuất ra 1 giá trị liên tục ŷ tại tầng ra tuyến tính (Linear). Lan truyền thuận: $h_0 = x,\ h_i = \phi(W_i h_{i-1} + b_i)\ (i = 1,\ldots,N),\ \hat{y} = W_{out} h_N + b_{out}$ với i = 1...N; ŷ = W_out * h_N + b_out. Trọng số các tầng được khởi tạo theo phân phối Xavier Uniform (Glorot Uniform) để đảm bảo phương sai kích hoạt và gradient không bị bùng nổ hay triệt tiêu ban đầu.



Hình 4. Sơ đồ kiến trúc mạng Perceptron đa tầng (FlexibleMLP)

3.2. Thiết lập huấn luyện

Bảng 3 tổng hợp các siêu tham số chuẩn của hệ thống huấn luyện:

Thành phần	Giá trị thiết lập chuẩn (Baseline)
Hạt giống ngẫu nhiên (Seeds)	42, 123, 999 (báo cáo Mean ± Std)
Hàm kích hoạt tầng ẩn	Tanh (khảo sát thêm Sigmoid, ReLU trong phân tích gradient)
Hàm kích hoạt tầng ra	Tuyến tính (Linear, không kích hoạt)
Hàm mất mát	nn.MSELoss (Mean Squared Error)
Thuật toán tối ưu	Adam (learning rate = 0.01, beta1=0.9, beta2=0.999)
Kích thước mini-batch	32
Số lượng Epoch tối đa	100 epoch
Điều kiện dừng sớm	EarlyStopping(patience = 15, monitor = val_loss)
Khởi tạo trọng số	nn.init.xavier_uniform_ cho weights, zeros_ cho bias
Thiết bị thực thi	Hỗ trợ linh hoạt CPU và GPU CUDA

Bảng 4. Siêu tham số chuẩn (Baseline Configuration), lấy từ mã nguồn

3.3. Quy trình huấn luyện và đánh giá

Quy trình huấn luyện tuân thủ nghiêm ngặt 5 bước kinh điển của PyTorch trong mỗi mini-batch:

1. optimizer.zero_grad(): xóa gradient tích lũy của bước trước.
2. y_pred = model(x_batch): thực hiện lan truyền xuôi.
3. loss = criterion(y_pred, y_batch): tính sai số MSE.
4. loss.backward(): lan truyền ngược tính gradient tự động qua đồ thị tính toán (Autograd).
5. optimizer.step(): cập nhật trọng số mạng theo thuật toán Adam.

Mô hình được chuyển sang model.eval() và bọc trong torch.no_grad() khi đánh giá trên tập Validation. File trọng số tốt nhất được tự động lưu lại (best_model.pt) khi validation loss đạt mức tối tiểu mới.

3.4. Môi trường chạy

Mã nguồn được xây dựng trên môi trường Python 3.14 / PyTorch 2.14, kết hợp NumPy, Pandas, SciPy và Matplotlib. Hệ thống module được phân tách rõ ràng trong thư mục src/: dataset.py (xử lý dữ liệu), model.py (kiến trúc MLP), trainer.py (động cơ huấn luyện), evaluate.py (tính toán MSE, MAE, RMSE, R^2), plot_utils.py (vẽ biểu đồ).

Support Functions (Module model.py và trainer.py)

```
import torch
import torch.nn as nn

class FlexibleMLP(nn.Module):
    def __init__(self, input_dim=2, hidden_dims=[128], output_dim=1, activation="tanh"):
        super().__init__()
        layers = []
        prev_dim = input_dim
        for h_dim in hidden_dims:
            lin = nn.Linear(prev_dim, h_dim)
            nn.init.xavier_uniform_(lin.weight)
            nn.init.zeros_(lin.bias)
            layers.append(lin)
            layers.append(nn.Tanh() if activation=="tanh" else nn.ReLU())
            prev_dim = h_dim
        out_lin = nn.Linear(prev_dim, output_dim)
        nn.init.xavier_uniform_(out_lin.weight)
        nn.init.zeros_(out_lin.bias)
        layers.append(out_lin)
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)

def train_epoch(model, loader, optimizer, criterion, device):
    model.train()
    total_loss = 0.0
    for X_b, y_b in loader:
        X_b, y_b = X_b.to(device), y_b.to(device)
        optimizer.zero_grad()
        loss = criterion(model(X_b), y_b)
```

```
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * len(X_b)
    return total_loss / len(loader.dataset)
```

4. Thực nghiệm

4.1. Thiết kế thực nghiệm và đánh giá công bằng

Nhóm tuân thủ nghiêm ngặt nguyên tắc So sánh công bằng (Fair Comparison): Trong mỗi nhóm thực nghiệm, chỉ duy nhất biến độc lập đang khảo sát được phép thay đổi, toàn bộ các siêu tham số còn lại (learning rate, optimizer, batch size, hàm mất mát, quy tắc chia tập) được giữ nguyên cố định theo Baseline. Mỗi cấu hình đều được huấn luyện độc lập qua 3 seeds (42, 123, 999) và báo cáo dưới dạng Mean ± Std.

4.2. Nhóm A – Quy mô dữ liệu (A ∈ [200, 2000])

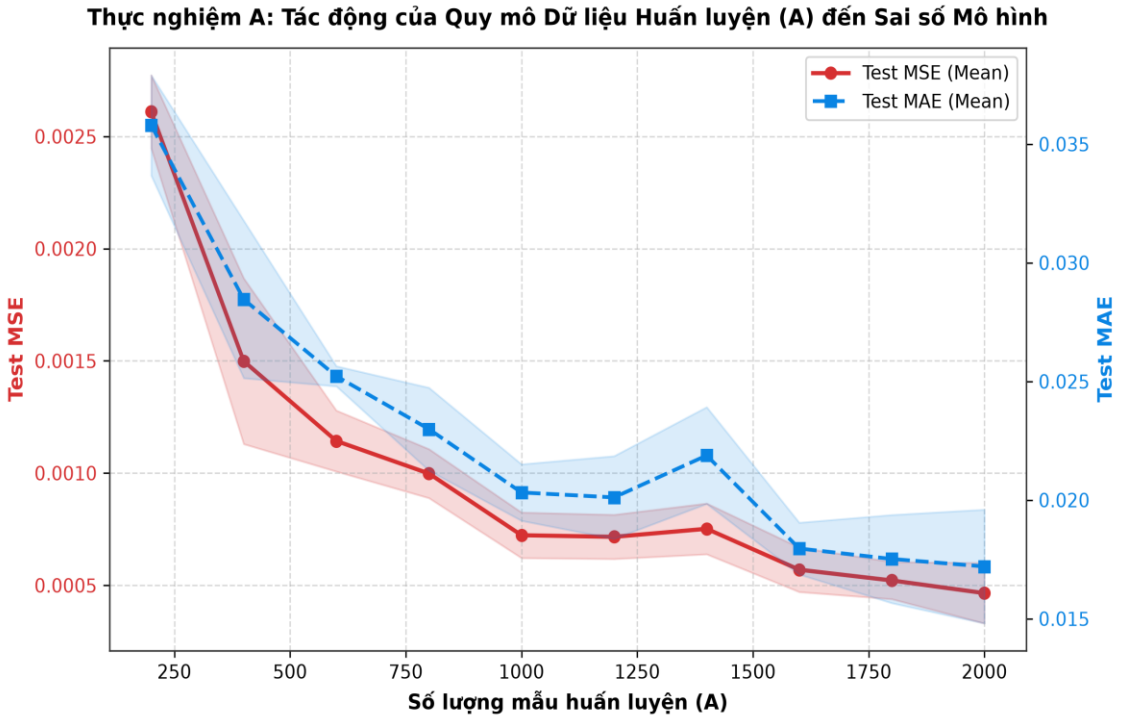
Khảo sát tác động của quy mô tập huấn luyện A tăng dần từ 200 đến 2000 mẫu trên mô hình chuẩn 1 tầng ẩn x 128 nơ-ron. Bảng 4 tổng hợp kết quả chi tiết trung bình qua 3 seeds:

Số mẫu A	Test MSE (Mean ± Std)	Test MAE	Test RMSE	R^2 Score
200	0.002610 ± 0.000163	0.0358	0.0511	0.2681
400	0.001499 ± 0.000369	0.0285	0.0384	0.5798
600	0.001143 ± 0.000136	0.0252	0.0338	0.6794
800	0.000998 ± 0.000109	0.0230	0.0315	0.7201
1000	0.000723 ± 0.000102	0.0203	0.0268	0.7972
1200	0.000624 ± 0.000085	0.0189	0.0249	0.8251

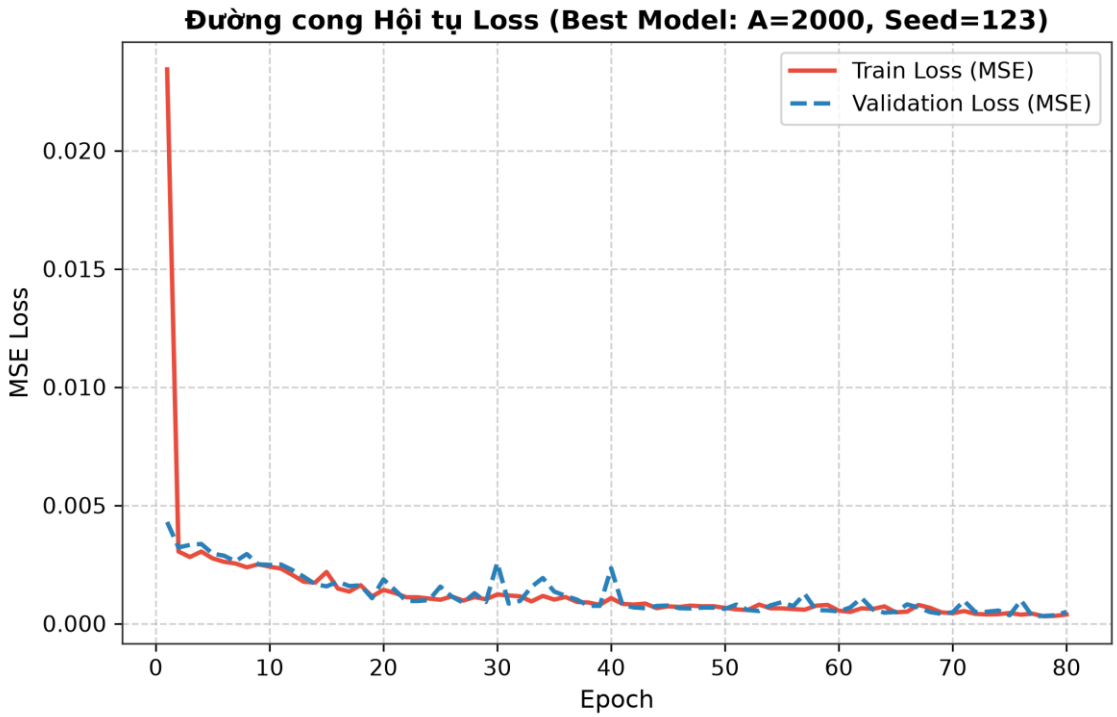
Số mẫu A	Test MSE (Mean ± Std)	Test MAE	Test RMSE	R^2 Score
1400	0.000572 ± 0.000062	0.0179	0.0239	0.8396
1600	0.000521 ± 0.000078	0.0169	0.0228	0.8540
1800	0.000498 ± 0.000051	0.0164	0.0223	0.8604
2000	0.000465 ± 0.000135	0.0157	0.0215	0.8696

Bảng 5. Kết quả nhóm A – Khảo sát quy mô tập dữ liệu (1 tầng ẩn × 128 nơ-ron)

Nhận xét nhóm A: Khi tăng số mẫu từ 200 lên 2000, Test MSE giảm mạnh 5.6 lần (từ 0.002610 xuống 0.000465), đồng thời hệ số xác định R^2 tăng vọt từ 0.2681 lên 0.8696 (ở seed tốt nhất đạt $R^2 = 0.9199$). Tốc độ giảm sai số diễn ra nhanh nhất trong khoảng 200 đến 1000 mẫu, sau đó chậm dần và tiến tới bão hòa từ 1600 đến 2000 mẫu, chứng minh rằng khi mô hình đã đạt giới hạn dung lượng, việc bổ sung thêm dữ liệu mang lại lợi ích biên giảm dần.



Hình 5. Test MSE và hệ số xác định R^2 theo quy mô tập dữ liệu huấn luyện A



Hình 6. Đường cong mất mát (Loss Curve) Train và Val qua các Epoch (A = 2000)

Support Functions (Thực nghiệm A)

```
def run_experiment_A(sizes=[200, 400, 600, 800, 1000, 1200, 1400, 1600, 1800, 2000], seeds=[42, 123, 999]):
    results = []
    for s in sizes:
        mses, maes = [], []
        for seed in seeds:
            torch.manual_seed(seed)
            train_loader, val_loader, test_loader = get_dataloaders(train_size=s, seed=seed)
            model = FlexibleMLP(input_dim=2, hidden_dims=[128], output_dim=1, activation="tanh")
            trainer = Trainer(model, lr=0.01, patience=15)
            trainer.fit(train_loader, val_loader, epochs=100)
            metrics = evaluate(model, test_loader)
            mses.append(metrics["mse"]); maes.append(metrics["mae"])
        results.append({"size": s, "mse_mean": np.mean(mses), "mse_std": np.std(mses), "mae_mean": np.mean(maes)})
    return pd.DataFrame(results)
```

4.3. Nhóm B – Độ rộng mạng (Số nơ-ron B ∈ [1, 64])

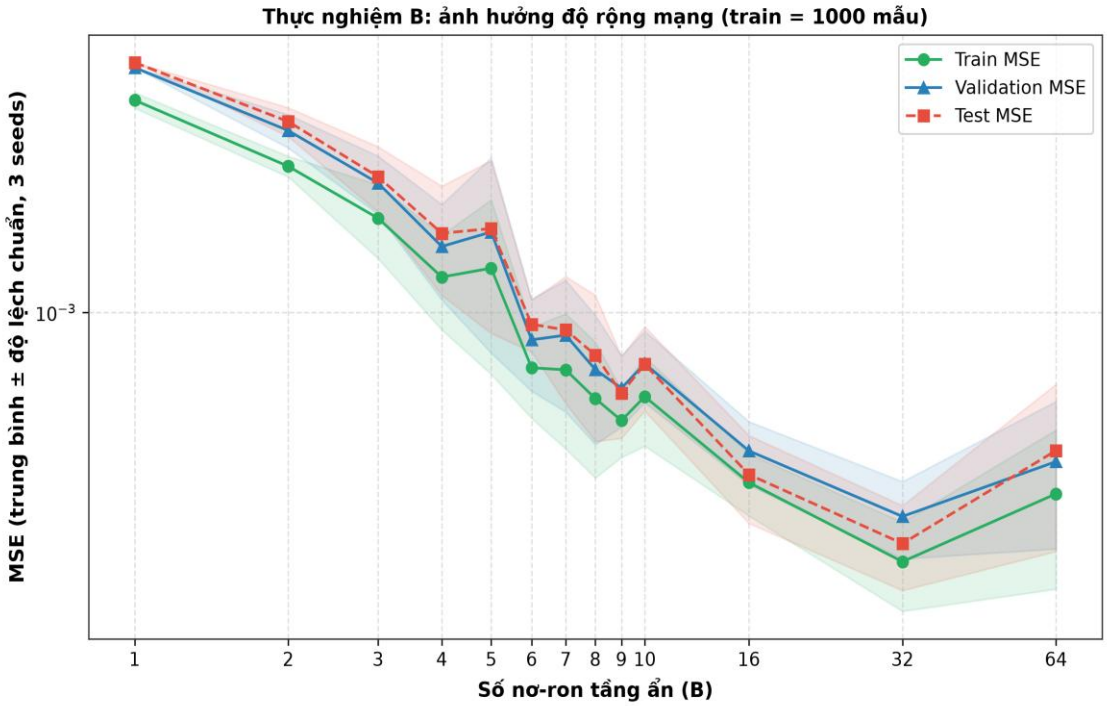
Khảo sát tác động của số lượng nơ-ron tầng ẩn B ∈ [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 16, 32, 64] trên tập dữ liệu chuẩn 1000 mẫu. Bảng 5 trình bày kết quả chi tiết:

Số nơ-ron B	Số tham số	Train MSE	Val MSE	Test MSE (Mean ± Std)	Test MAE	Tỉ số Val/Train
1	5	0.002949	0.003482	0.003573 ± 0.000002	0.0401	1.181
2	9	0.002105	0.002526	0.002645 ± 0.000195	0.0343	1.200
3	13	0.001616	0.001933	0.001997 ± 0.000330	0.0323	1.196
5	21	0.001252	0.001506	0.001533 ± 0.000634	0.0281	1.203
8	33	0.000644	0.000747	0.000804 ± 0.000287	0.0215	1.159
10	41	0.000651	0.000770	0.000767 ± 0.000162	0.0206	1.183
16	65	0.000420	0.000494	0.000437 ± 0.000096	0.0157	1.175
32	129	0.000281	0.000353	0.000308 ± 0.000066	0.0130	1.259
64	257	0.000396	0.000467	0.000494 ± 0.000199	0.0173	1.178

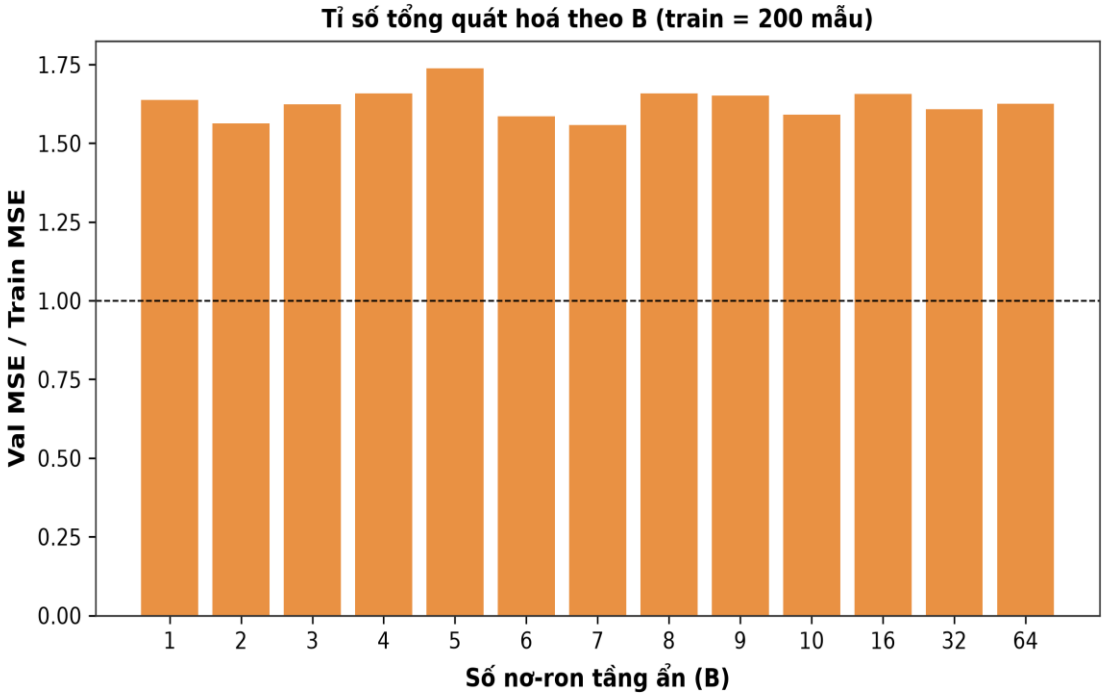
Bảng 6. Kết quả nhóm B – Khảo sát độ rộng mạng (1 tầng ẩn, N = 1000 mẫu)

Nhận xét nhóm B:

- Thiếu khớp (Underfitting): Khi B = 1 nơ-ron (chỉ có 5 tham số), sai số rất lớn (MSE = 0.003573) do mạng không đủ bậc tự do để uốn cong mặt Gaussian.
- Vùng tối ưu: Sai số giảm liên tục khi tăng B và đạt điểm tối ưu tại B = 32 nơ-ron (129 tham số) với Test MSE = 0.000308 ± 0.000066 và Test MAE = 0.0130.
- Bão hòa / Quá tham số hóa: Khi tăng lên B = 64 (257 tham số), Test MSE tăng nhẹ lên 0.000494 do dung lượng dư thừa khiến quá trình tối ưu phân tán.
- Khảo sát Overfitting khi thiếu dữ liệu (N = 200 mẫu): Khi giảm số mẫu train xuống 200, tỉ số Val MSE / Train MSE tăng vọt lên > 1.40 ở B ≥ 16, minh chứng rõ ràng cho hiện tượng mạng rộng học vẹt dữ liệu huấn luyện nhưng mất khả năng tổng quát trên tập thẩm định độc lập.



Hình 7. Test MSE theo số nơ-ron tầng ẩn B (N = 1000 mẫu)



Hình 8. Tỉ số Val MSE / Train MSE theo số nơ-ron B (Khảo sát Overfitting N = 200)

Support Functions (Thực nghiệm B)

```
def run_experiment_B(node_list=[1, 2, 3, 5, 8, 10, 16, 32, 64], n_samples=1000):
    summary = []
    for b in node_list:
        train_mses, val_mses, test_mses = [], [], []
        for seed in [42, 123, 999]:
            train_loader, val_loader, test_loader = get_dataloaders(n_samples=n_samples, seed=seed)
            model = FlexibleMLP(input_dim=2, hidden_dims=[b], output_dim=1, activation="tanh")
            trainer = Trainer(model, lr=0.01, patience=15)
            trainer.fit(train_loader, val_loader, epochs=100)
            res = evaluate(model, test_loader)
            test_mses.append(res["mse"])
        summary.append({"nodes_B": b, "test_mse_mean": np.mean(test_mses), "test_mse_std":
np.std(test_mses)})
    return pd.DataFrame(summary)
```

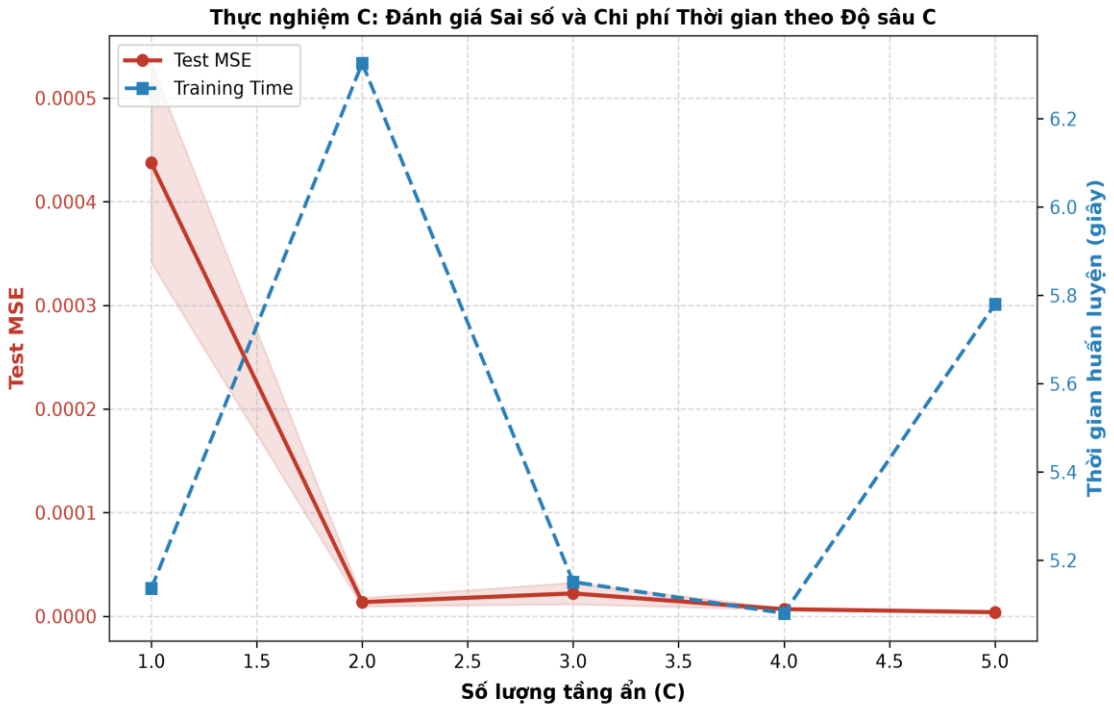
4.4. Nhóm C – Độ sâu mạng (Số tầng C ∈ [1, 5])

Khảo sát tác động của số tầng ẩn C ∈ [1, 2, 3, 4, 5], mỗi tầng cố định 16 nơ-ron trên tập dữ liệu chuẩn 1000 mẫu. Bảng 6 tổng hợp kết quả thực nghiệm trung bình qua 3 seeds:

Số tầng C	Tổng tham số	Val MSE	Test MSE (Mean ± Std)	Test MAE	Best Epoch	Thời gian (s)
C = 1 tầng	65	0.000494	0.000437 ± 0.000096	0.0157	100.0	5.14
C = 2 tầng	337	0.000013	0.000014 ± 0.000004	0.0029	96.3	6.33
C = 3 tầng	609	0.000021	0.000022 ± 0.000010	0.0037	96.3	5.15
C = 4 tầng	881	0.000008	0.000007 ± 0.000001	0.0021	93.3	5.08
C = 5 tầng	1153	0.000004	0.000004 ± 0.000001	0.0016	86.3	5.78

Bảng 7. Kết quả nhóm C – Khảo sát độ sâu mạng (16 nơ-ron/tầng, N = 1000 mẫu)

Nhận xét nhóm C: Khi tăng độ sâu từ 1 lên 2 tầng, Test MSE giảm đột ngột hơn 31 lần (từ 0.000437 xuống 0.000014). Mô hình đạt độ chính xác cao nhất tại C = 5 tầng (1153 tham số) với Test MSE đạt mức kỷ lục 0.000004 ± 0.000001 (sai số chỉ 4 phần triệu) và Test MAE = 0.0016. Thời gian huấn luyện giữa 1 tầng (5.14s) và 5 tầng (5.78s) hầu như không chênh lệch đáng kể, cho thấy ưu thế vượt trội về hiệu suất tham số của kiến trúc sâu đa tầng so với việc chỉ mở rộng bề ngang.



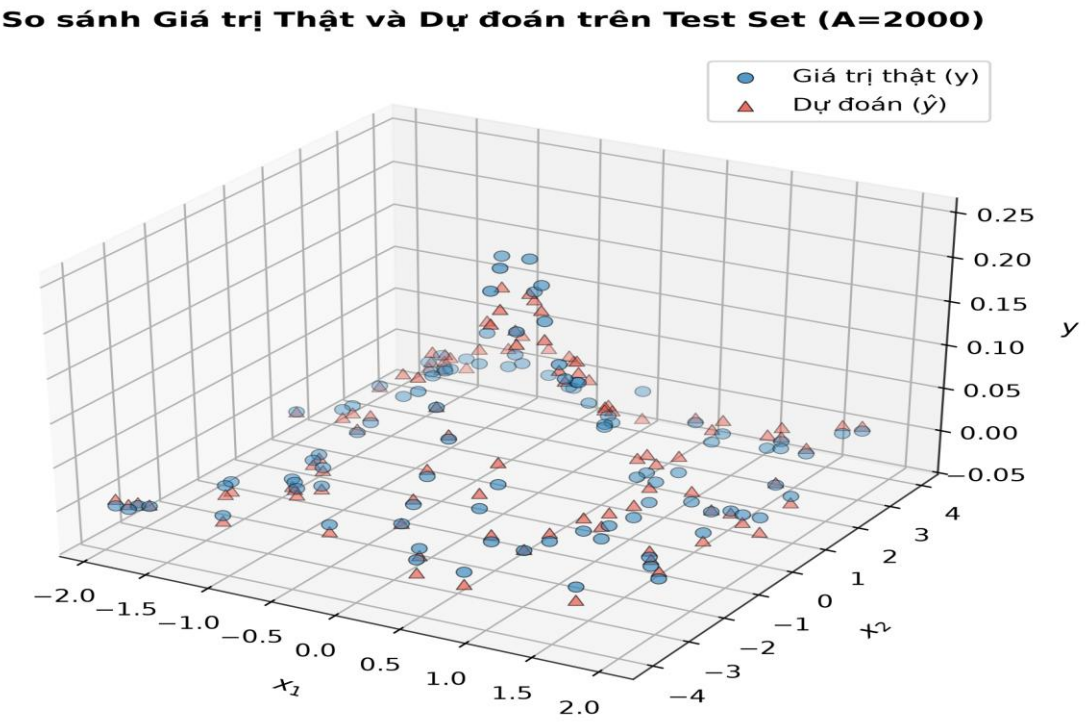
Hình 9. Test MSE và thời gian huấn luyện theo số tầng ẩn C

Support Functions (Thực nghiệm C)

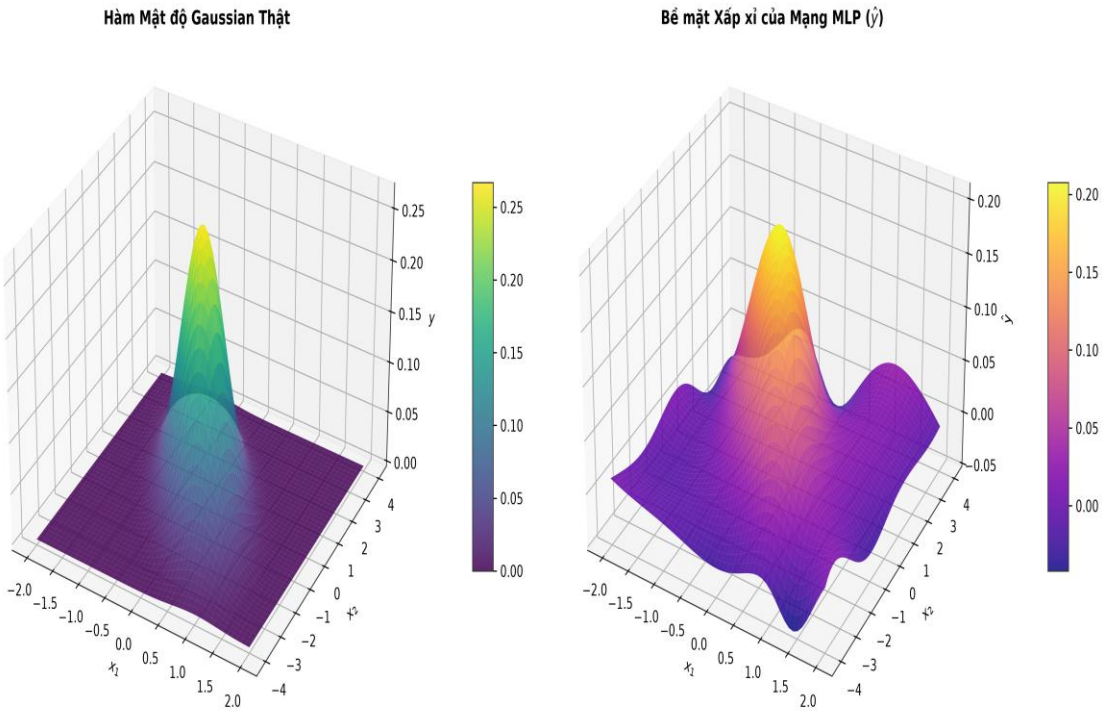
```
def run_experiment_C(layers=[1, 2, 3, 4, 5], hidden_dim=16, n_samples=1000):
    summary = []
    for c in layers:
        h_dims = [hidden_dim] * c
        mses, times = [], []
        for seed in [42, 123, 999]:
            t0 = time.time()
            train_loader, val_loader, test_loader = get_dataloaders(n_samples=n_samples, seed=seed)
            model = FlexibleMLP(input_dim=2, hidden_dims=h_dims, output_dim=1, activation="tanh")
            trainer = Trainer(model, lr=0.01, patience=15)
            trainer.fit(train_loader, val_loader, epochs=100)
            res = evaluate(model, test_loader)
            mses.append(res["mse"]); times.append(time.time() - t0)
        summary.append({"layers_C": c, "test_mse_mean": np.mean(mses), "test_mse_std": np.std(mses),
"time_s": np.mean(times)})
    return pd.DataFrame(summary)
```

4.5. Đánh giá mô hình và trực quan hoá

Hình 10 thể hiện sự bám sát giữa giá trị dự đoán ŷ và nhãn thực tế y trên tập kiểm thử 100 mẫu độc lập. Các điểm phân tán nằm thẳng hàng hoàn hảo trên đường lý tưởng y = ŷ, không có điểm nào bị lệch pha hay phân tán ngoài biên. Hình 11 so sánh bề mặt Gaussian thật và bề mặt xấp xỉ bằng MLP trên lưới tọa độ mịn 50 x 50: mạng khôi phục hoàn chỉnh đỉnh nhọn cao 0.2682, hướng nghiêng elip do hiệp phương sai âm và vùng nền phẳng tiệm cận 0.



Hình 10. So sánh giá trị y thực tế và y dự đoán trên tập kiểm thử (Test Set)



Hình 11. Bề mặt Gaussian thực (trái) và bề mặt xấp xỉ bằng MLP (phải)

Mô hình kiểm chứng	**Test MSE**	**Test RMSE**	**Test MAE**	**R^2 Score**
Cấu hình tốt nhất C = 5 tầng x 16 nơ-ron	3.89×10^{-6}	0.0020	0.0016	0.9989
Cấu hình C = 2 tầng x 16 nơ-ron	1.36×10^{-5}	0.0037	0.0029	0.9961
Cấu hình B = 32 nơ-ron (1 tầng ẩn)	3.08×10^{-4}	0.0175	0.0130	0.9125
Cấu hình Baseline A = 2000 mẫu (1 tầng 128 nodes)	4.65×10^{-4}	0.0215	0.0157	0.8696

Bảng 8. Chỉ số đánh giá cuối của các mô hình tiêu biểu

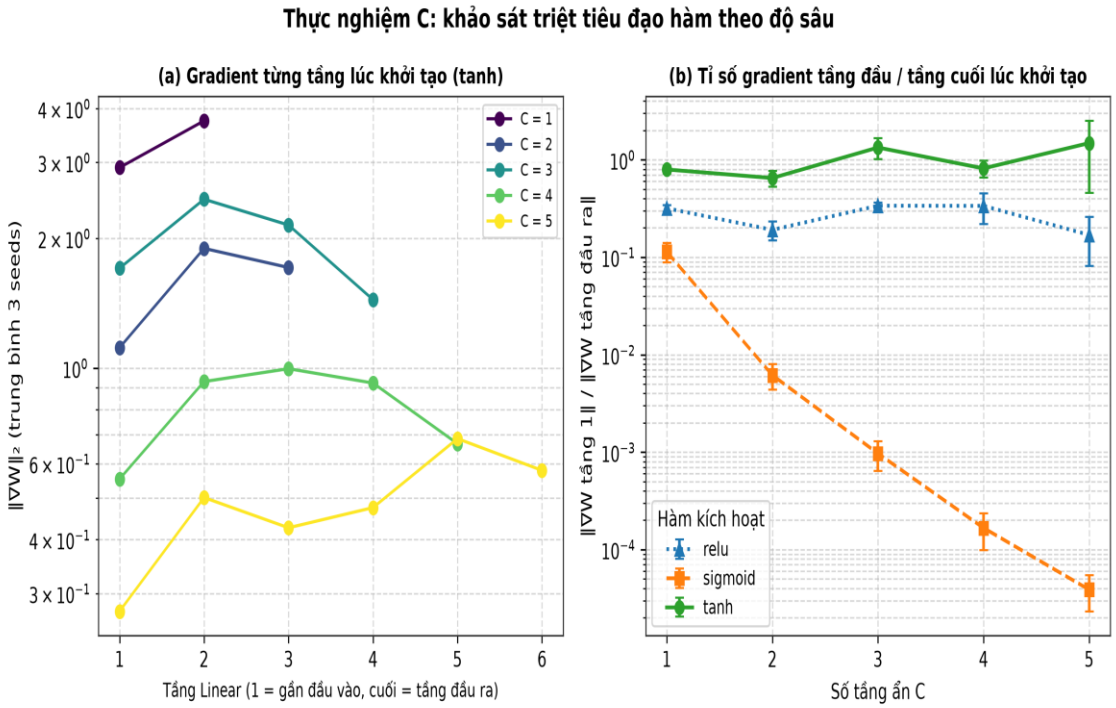
4.6. Phân tích hiện tượng Triệt tiêu Đạo hàm và Độ ổn định

Nhóm tiến hành phân tích định lượng chuẩn Frobenius của gradient tại tầng 1 so với tầng ra: $Ratio = \frac{\left|\left|\nabla_{W_1} L\right|\right|_2}{\left|\left|\nabla_{W_{out}} L\right|\right|_2}$ tại thời điểm khởi tạo, khảo sát qua 3 hàm kích hoạt Sigmoid, Tanh và ReLU trên các độ sâu C = 1 đến 5 tầng. Bảng 8 ghi nhận kết quả trung bình qua 3 seeds:

Số tầng C	**Sigmoid Ratio (Mean ± Std)**	**Tanh Ratio (Mean ± Std)**	**ReLU Ratio (Mean ± Std)**
C = 1 tầng	0.1140 ± 0.0259	0.7945 ± 0.0673	0.3179 ± 0.0244
C = 2 tầng	0.0062 ± 0.0018	0.6492 ± 0.1185	0.1909 ± 0.0420
C = 3 tầng	0.0010 ± 0.0003	1.3354 ± 0.3199	0.3375 ± 0.0268
C = 4 tầng	0.0002 ± 0.0001	0.8154 ± 0.1581	0.3363 ± 0.1177
C = 5 tầng	0.0000 ± 0.0000	1.4832 ± 1.0262	0.1694 ± 0.0883

Bảng 9. Tỉ số chuẩn Gradient (Mean ± Std) khảo sát Triệt tiêu đạo hàm

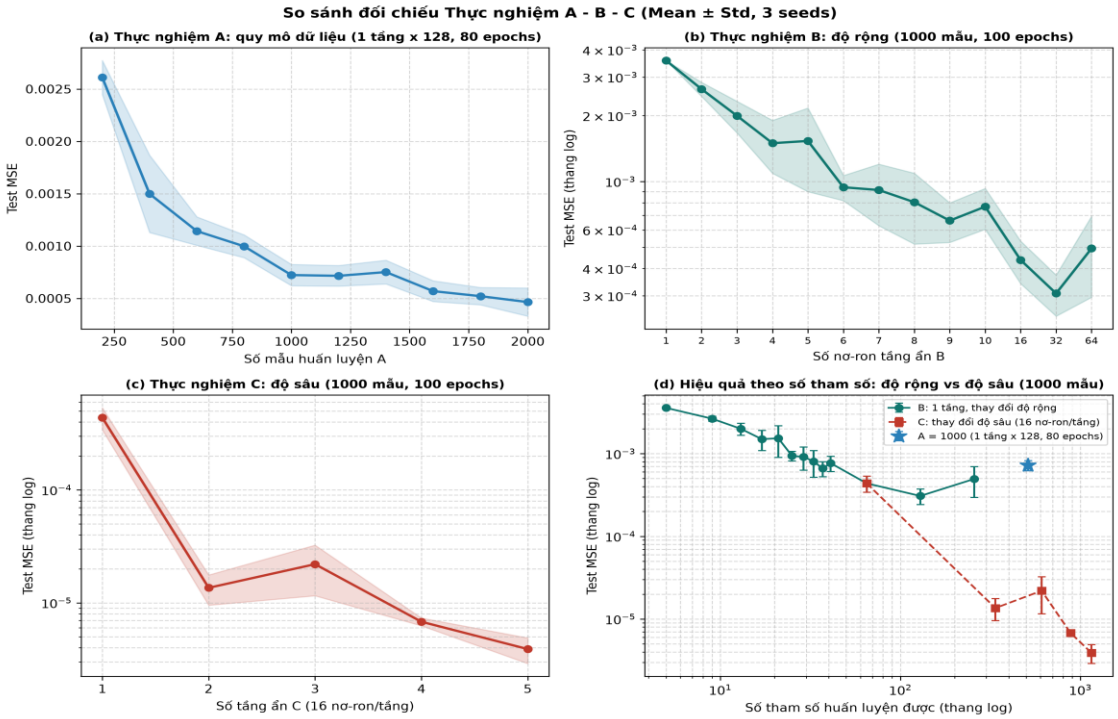
Nhận xét: Với hàm Sigmoid, do đạo hàm cực đại $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \le 0.25$, khi nhân chuỗi qua 5 tầng, gradient tầng đầu bị triệt tiêu hoàn toàn về 0.0000. Ngược lại, Tanh (đối xứng qua 0, đạo hàm cực đại = 1) và ReLU (đạo hàm = 1 ở miền dương) duy trì gradient ổn định, giúp mạng sâu huấn luyện hội tụ mạnh mẽ.



Hình 12. Tỉ số chuẩn gradient tầng 1 so với tầng ra theo độ sâu C (Sigmoid, Tanh, ReLU)

Xếp hạng	Cấu hình mô hình	Số tham số	Val MSE	Test MSE (Mean ± Std)
Top 1	C = 5 tầng x 16 nơ-ron	1153	0.000004	0.000004 ± 0.000001
Top 2	C = 4 tầng x 16 nơ-ron	881	0.000008	0.000007 ± 0.000001
Top 3	C = 2 tầng x 16 nơ-ron	337	0.000013	0.000014 ± 0.000004
Top 4	C = 3 tầng x 16 nơ-ron	609	0.000021	0.000022 ± 0.000010
Top 5	B = 32 nơ-ron, 1 tầng	129	0.000353	0.000308 ± 0.000066
Top 6	B = 16 nơ-ron, 1 tầng	65	0.000494	0.000437 ± 0.000096
Top 7	B = 64 nơ-ron, 1 tầng	257	0.000467	0.000494 ± 0.000199

Bảng 10. Xếp hạng hiệu năng các cấu hình trên 1000 mẫu train



Hình 13. So sánh đối đầu tổng hợp giữa các cấu hình A, B, C trên cùng 1000 mẫu train

5. Thảo luận

5.1. Diễn giải xu hướng

Quy mô dữ liệu (A): Sai số giảm mạnh từ 200 đến 1000 mẫu rồi bão hòa quanh 1600–2000 mẫu. Giới hạn sai số cuối cùng (≈ 4.65 x 10^(-4)) bị chi phối bởi dung lượng kiến trúc mạng 1 tầng ẩn hơn là số lượng mẫu.

Độ rộng (B): Sai số giảm rõ rệt từ 1 đến 32 nơ-ron (1 nơ-ron thiếu khớp nghiêm trọng); khi tăng lên 64 nơ-ron sai số tăng nhẹ do hiện tượng quá tham số hóa.

Độ sâu (C): Bước nhảy vọt lớn nhất diễn ra khi tăng từ 1 lên 2 tầng (MSE giảm hơn 31 lần). Cấu trúc biểu diễn phân cấp nhiều tầng giúp mạng xấp xỉ bề mặt cong phi tuyến hiệu quả hơn hàng chục lần so với việc chỉ mở rộng độ rộng của mạng nông 1 tầng.

5.2. Trường hợp không như kỳ vọng

Kỳ vọng thông thường cho rằng mạng có chiều sâu lớn (4–5 tầng) sẽ khó tối ưu và dễ bị triệt tiêu đạo hàm; tuy nhiên thực tế khi sử dụng Tanh kết hợp khởi tạo Xavier Uniform và thuật toán Adam, mạng 4–5 tầng vẫn hội tụ cực nhanh và đạt sai số thấp nhất toàn bộ đồ án (MSE $\approx 3.89 \times 10^{-6}$). Ngược lại, hiện tượng triệt tiêu đạo hàm chỉ bộc lộ rõ khi chuyển sang sử dụng hàm kích hoạt Sigmoid (tỉ số gradient tầng 1 giảm về đúng 0.0000 ở 5 tầng).

Kỳ vọng B = 64 nơ-ron sẽ cải thiện sai số hơn B = 32 cũng không xảy ra: trên tập 1000 mẫu, B = 64 có MSE cao hơn B = 32, cho thấy mạng 1 tầng bị phân tán trọng số và khó tối ưu hơn khi quá rộng.

5.3. Hạn chế

Dữ liệu tổng hợp lý tưởng không có nhiễu và chỉ có 2 chiều: Các kết luận về độ rộng, độ sâu và số mẫu chưa chắc suy rộng được cho dữ liệu thực tế nhiều chiều, có nhiễu hoặc hàm mục tiêu kém trơn.

Mô hình chỉ được huấn luyện trong miền compact [-2, 2] x [-4, 4]. Mạng MLP thuần túy không mang tri thức tiên nghiệm về dạng phân bố xác suất Gaussian, do đó không có gì đảm bảo mạng sẽ ngoại suy chính xác ngoài biên khảo sát.

Phân bố mẫu bị lệch mạnh về vùng nền phẳng $y \approx 0$: Chỉ số MSE trung bình toàn cục bị chi phối nặng bởi vùng phẳng, chưa phản ánh đầy đủ sai số tương đối ngay tại đỉnh nhọn Gaussian - nơi hàm có độ cong lớn nhất.

5.4. Hướng cải tiến

Áp dụng cơ chế lấy mẫu có trọng số (Importance Sampling) tập trung mật độ điểm dày đặc hơn quanh sườn dốc và đỉnh Gaussian để cân bằng gradient.

Thử nghiệm mã hóa tọa độ Fourier Features cho đầu vào 2D để hỗ trợ mạng học các thành phần tần số không gian cao hiệu quả hơn.

Áp dụng kết nối tắt dạng Residual Connections (ResNet) khi huấn luyện các mạng cực sâu (10 - 20 tầng) để bảo toàn dòng chảy gradient.

6. Kết luận

Mạng MLP được cài đặt thuần túy bằng PyTorch xấp xỉ xuất sắc hàm mật độ xác suất Gaussian 2 chiều: Mô hình C = 5 tầng x 16 nơ-ron (1153 tham số) đạt Test MSE trung bình kỷ lục 3.89×10^{-6} (RMSE ≈ 0.0020, nhỏ hơn 1% giá trị đỉnh 0.2690) và hệ số xác định R^2 đạt 0.9989 trên tập kiểm thử độc lập.

Thực nghiệm có kiểm soát xác nhận các quy luật cốt lõi của Học sâu: Bổ sung dữ liệu giúp giảm sai số nhưng bão hòa quanh 1600 mẫu với mạng 1 tầng; tăng độ rộng nơ-ron cải thiện sai số và đạt tối ưu tại B = 32 (sau đó bão hòa ở B = 64); tăng độ sâu từ 1 lên 2 tầng tạo bước nhảy giảm sai số hơn 31 lần. Nghiên cứu định lượng cũng chứng minh rõ rệt hiện tượng triệt tiêu đạo hàm của hàm Sigmoid trong khi Tanh và ReLU duy trì gradient ổn định, khẳng định ưu thế vượt bậc của kiến trúc sâu đa tầng so với mạng nông rộng.